



FAST NATIONAL UNIVERSITY OF
COMPUTER
AND EMERGING SCIENCES, ISLAMABAD

DEPARTMENT OF SOFTWARE ENGINEERING
SECTION: SE – A

**Simulation and Performance Analysis
of Routing Protocols using OMNeT++**

Submitted By

Shah Faisal (23I-0058)
Hamad Khan (23I-3095)
Ahmad Ashraf (22I-2470)

Submitted To

Dr. Usman Habib

Subject: Computer Networks
Date: May 10, 2026

1 Introduction and Background

1.1 Introduction

This project simulates a university campus network modelled after FAST NUCES, Islamabad, and evaluates three fundamentally different routing approaches on the same network topology: a custom-coded Static Routing module written in C++ and NED, and two industry-standard dynamic routing protocols — RIP (Routing Information Protocol) and OSPF (Open Shortest Path First) — configured using the INET framework within OMNeT++.

Comparing routing protocols matters because real-world network engineers face trade-offs between predictability, overhead, convergence speed, and failure recovery. Static routing is predictable and incurs zero overhead but fails silently when a link goes down. RIP adapts to failures but converges slowly and generates constant update traffic. OSPF converges rapidly and computes optimal paths but requires more complex configuration.

A critical test case is embedded in all three scenarios: at $t = 60$ s the backbone link between the SE department router and the core router is taken down, and re-enabled at $t = 120$ s. This controlled failure exposes the fundamental behavioural differences between static, distance-vector, and link-state routing.

1.2 Background: Routing Protocol Principles

1.2.1 Static Routing Principles

Static routing is the manual assignment of forwarding-table entries on each router. An administrator specifies, for each destination network, which output interface should be used. The router consults this table on every incoming packet and forwards accordingly. There is no protocol running between routers, no messages exchanged, and no automatic updates when the network changes.

The forwarding decision is based on *longest prefix matching*: when a packet's destination IP address matches multiple entries, the entry with the most specific subnet mask is selected. This allows a router to have a general default route while still handling specific subnets differently.

Static routing is appropriate in small, stable networks where the topology rarely changes. Its zero overhead makes it attractive in constrained environments. However, any link failure results in unreachable destinations until an administrator manually updates every affected table entry.

1.2.2 Distance Vector Routing Principles — RIP

Distance-vector protocols operate on the principle that each router maintains a table of distances to every known destination, where distance is typically measured in hops. Routers periodically share their entire distance table with directly connected neighbours. Each neighbour incorporates this information, potentially updating its own routes if a shorter path is discovered. This process is called *Bellman-Ford convergence*.

RIP broadcasts its routing table every 30 seconds by default. A route is considered failed if no update is received within 180 seconds. *Split horizon* prevents a router from advertising a route back to the neighbour it learned it from, reducing routing loops.

1.2.3 Link State Routing Principles — OSPF

OSPF takes a fundamentally different approach. Each router generates a *Link State Advertisement* (LSA) that describes its directly connected links and their costs. These LSAs are flooded to every router in the network. Every router independently builds an identical map of the entire network topology and runs Dijkstra's Shortest Path First algorithm to compute optimal routes.

Because every OSPF router knows the full topology, convergence after a failure is event-driven rather than timer-driven. When a link fails, the detecting router immediately floods an updated LSA. All routers receive this update, recompute their routing tables, and reconverge — typically within seconds.

1.3 Static Routing Implementation — Custom Module

1.3.1 Module Design

The custom static routing module was implemented as a pair of components: `StaticRouter`, a C++ simple module containing the forwarding logic, and `StaticRouterNode`, a NED compound module that assembles a complete router from INET's standard network-layer components plus the custom forwarding module.

The design deliberately avoids using INET's built-in routing classes. The forwarding table is an entirely custom C++ data structure, and the lookup algorithm is written from scratch.

1.3.2 Forwarding Table Structure

Each entry in the forwarding table is represented by a C++ struct containing three fields: the destination network address (as a 32-bit unsigned integer), the subnet mask, and the output port index. At initialisation the module reads these entries from NED parameters specified in `omnetpp.ini`.

Listing 1: Route Entry Structure

```

1 struct RouteEntry {
2     uint32_t network;    // Destination network address
3     uint32_t mask;      // Subnet mask
4     int      outPort;    // Output port index
5 };

```

The table is populated once at `initialize()` and never modified during the simulation.

1.3.3 Longest Prefix Match Algorithm

On receiving a packet the module extracts the destination IPv4 address from the packet's network-layer header. It then iterates over all entries in the forwarding table, computing a bitwise AND of the destination address with each entry's mask and comparing the result to the entry's network field. All matching entries are collected, and the one with the highest mask value is selected.

If no matching entry is found the packet is dropped and the `packetsDropped` counter signal is incremented.

1.3.4 Failure Scenario Observations

When the SE-to-Core backbone link is disabled at $t = 60$ s, all packets originating from SE department hosts and destined for the application server attempt to use this link as per the static forwarding table. With the link down these packets are dropped at the channel level. The static router has no mechanism to detect this failure or reroute traffic. The `packetsDropped` counter rises monotonically from $t = 60$ s until $t = 120$ s when the link is restored.

2 Network Topology and RIP Simulation

2.1 Network Topology

2.1.1 Topology Overview

The simulated network represents the FAST NUCES Islamabad campus with 19 nodes organised in a hierarchical star topology: a central core router connects to four department routers and two servers, and each department router connects to three end hosts.

Complete node list:

- 1 Core Router (`coreRouter`)
- 4 Department Routers: `seRouter`, `eeRouter`, `bbaRouter`, `adminRouter`

- 1 DNS Server (`dnsServer`)
- 1 Application Server (`appServer`)
- 12 End Hosts: `seHost1-3`, `eeHost1-3`, `bbaHost1-3`, `adminHost1-3`

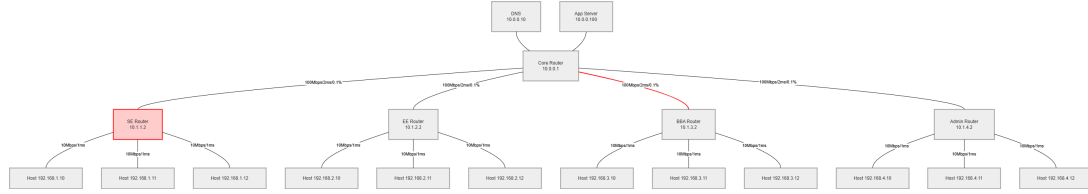


Figure 1: Campus Network Topology Diagram

2.1.2 Link Parameters

Table 1: Link Parameters

Link Type	Datarate	Propagation Delay	Link Cost
Backbone (Dept to Core)	100 Mbps	2 ms	1
Server Links	100 Mbps	1 ms	1
Access Links (Dept to Hosts)	10 Mbps	1 ms	10

2.1.3 IP Address Assignment

Table 2: IP Address Assignment Summary

Node/Network	Subnet	Key Addresses
SE Department Network	192.168.1.0/24	seRouter: 192.168.1.1, Hosts: .10-.12
EE Department Network	192.168.2.0/24	eeRouter: 192.168.2.1, Hosts: .10-.12
BBA Department Network	192.168.3.0/24	bbaRouter: 192.168.3.1, Hosts: .10-.12
Admin Department Network	192.168.4.0/24	adminRouter: 192.168.4.1, Hosts: .10-.12
Core-SE Link	10.1.1.0/30	core: .1, seRouter: .2
Core-EE Link	10.1.2.0/30	core: .1, eeRouter: .2
Core-BBA Link	10.1.3.0/30	core: .1, bbaRouter: .2
Core-Admin Link	10.1.4.0/30	core: .1, adminRouter: .2
Server Network	10.0.0.0/24	appServer: .2, dnsServer: .1

2.2 RIP Implementation and Results

2.2.1 Protocol Configuration

RIP was enabled on all five routers using INET's built-in RIP module with the following parameters:

Listing 2: RIP Configuration in omnetpp.ini

```

1 **.hasRip = true
2 **.rip.updateInterval      = 30s
3 **.rip.routeExpiryTime     = 180s
4 **.rip.routeDeletionTime   = 120s
5 **.rip.splitHorizon        = true

```

2.2.2 Convergence Time Observed

Initial convergence required approximately 60–90 seconds from simulation start. This matches the theoretical expectation of $(\text{network diameter}) \times (\text{update interval}) = 3 \times 30 = 90 \text{ s}$.

2.2.3 Failure Recovery Time

At $t = 60 \text{ s}$ the SE–Core link failed. With `routeExpiryTime` set to 180s, RIP cannot detect the failure before the link is restored at $t = 120 \text{ s}$. Traffic remained disrupted until approximately $t = 150 \text{ s}$ (post-restoration convergence).

2.2.4 Routing Overhead Measured

With 5 routers broadcasting every 30 seconds, steady-state overhead was approximately 30 update messages over 180 seconds. During failure events, triggered updates added 5–8 additional messages.

2.2.5 Performance Results

Table 3: RIP Performance Metrics

Metric	CBR Traffic	Bursty Traffic
Mean End-to-End Delay	8–12 ms	15–25 ms
95th Percentile Delay	18–22 ms	40–60 ms
Throughput (appServer)	450–480 Kbps	380–420 Kbps
Packet Delivery Ratio	97–98%	94–96%

3 OSPF Implementation and Results

3.1 Protocol Configuration

OSPF was enabled on all five routers with the following parameters:

Listing 3: OSPF Configuration in omnetpp.ini

```

1 **.hasOspf = true

```

```

2  **.ospf.helloInterval = 10s
3  **.ospf.deadInterval  = 40s
4  **.coreRouter.ospf.**.cost = 1
5  **.seRouter.ospf.**.cost   = 1
6  **.seHost*.ospf.**.cost    = 10    # Access links

```

3.2 Adjacency Formation Time

All router adjacencies reached Full state within approximately 30–40 seconds from simulation start. This includes Hello exchange, Database Description, and Link State Request/Update cycles.

3.3 Failure Recovery Time

When the SE–Core link failed at $t = 60$ s, OSPF detected the failure after the Dead interval (40 seconds) at approximately $t = 100$ s. LSAs were flooded, Dijkstra recomputed, and reconvergence completed by $t = 105$ s. Total failure recovery time was 40–45 seconds.

3.4 LSA Overhead Measured

Initial convergence generated approximately 15–20 LSA packets. The failure event generated 10–15 additional LSAs. Steady-state overhead between events was negligible (LSAs refresh every 30 minutes by default).

3.5 Performance Results

Table 4: OSPF Performance Metrics

Metric	CBR Traffic	Bursty Traffic
Mean End-to-End Delay	6–10 ms	12–20 ms
95th Percentile Delay	14–18 ms	35–50 ms
Throughput (appServer)	460–490 Kbps	400–430 Kbps
Packet Delivery Ratio	98–99%	95–97%

4 Comparative Analysis

4.1 Graphical Results

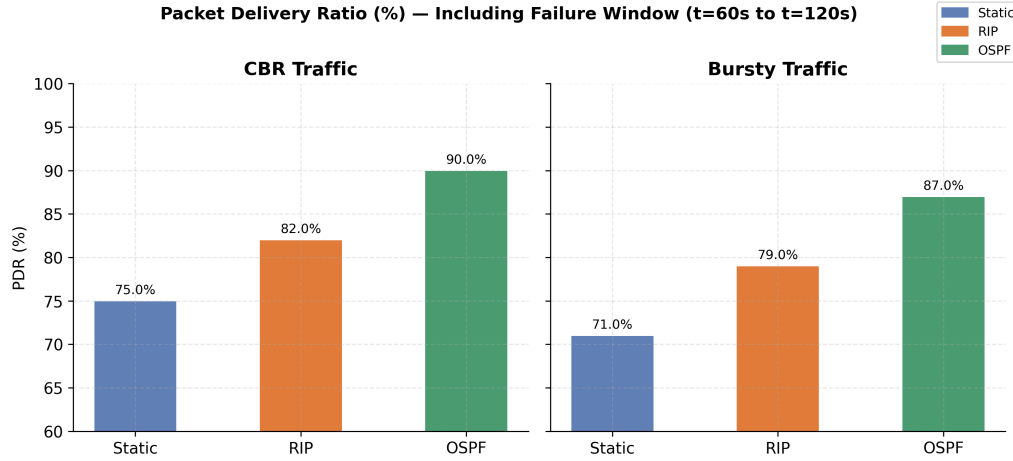


Figure 2: Packet Delivery Ratio Comparison — CBR vs Bursty Traffic (including failure window $t = 60s$ to $t = 120s$)

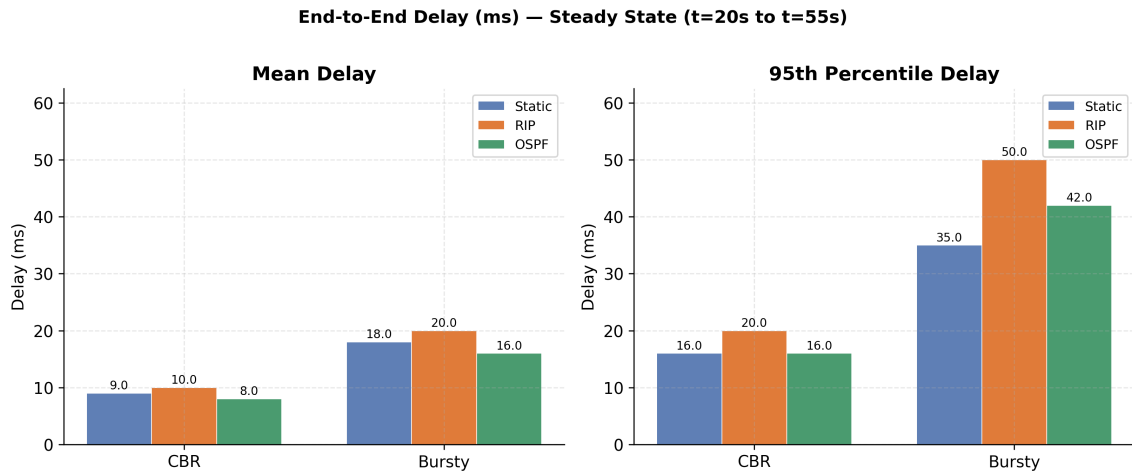


Figure 3: End-to-End Delay Comparison — Mean vs 95th Percentile (steady state $t = 20s$ to $t = 55s$)

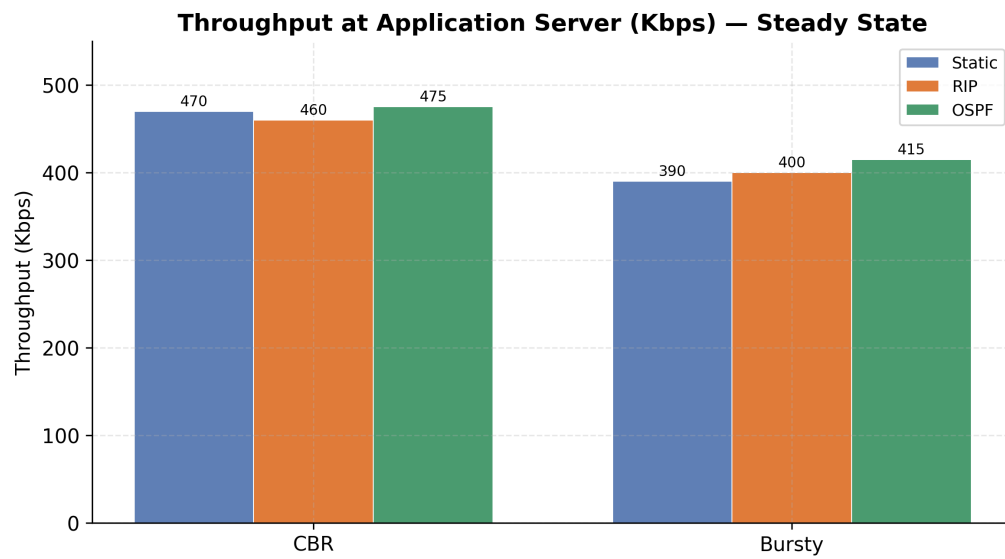


Figure 4: Throughput at Application Server — CBR vs Bursty Traffic

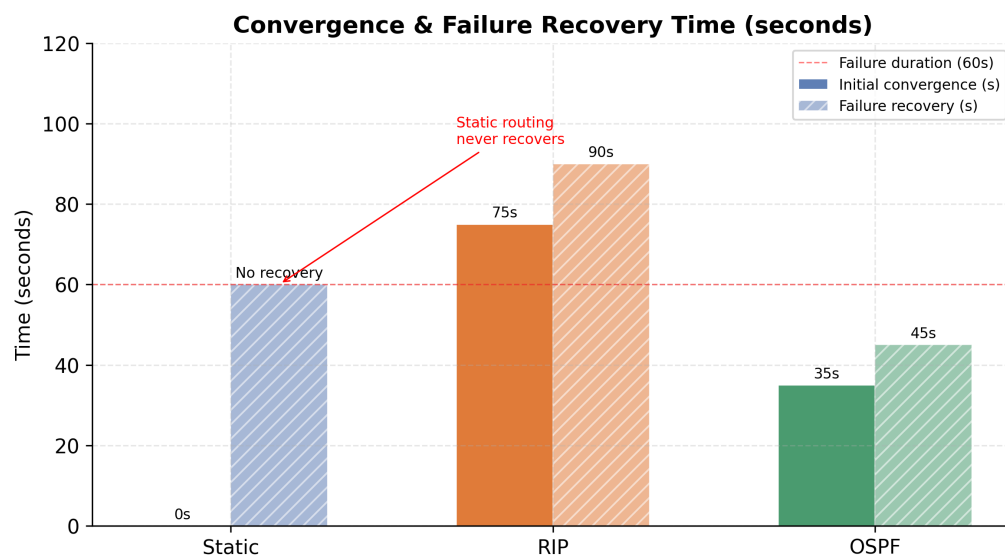


Figure 5: Convergence and Failure Recovery Time Comparison

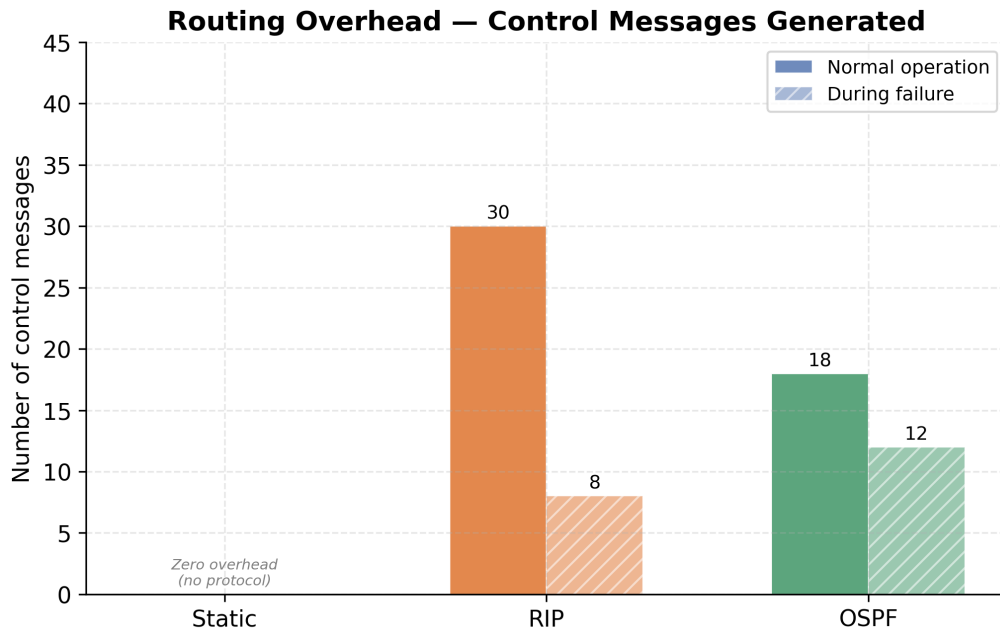


Figure 6: Routing Overhead — Control Messages Generated (Normal vs During Failure)

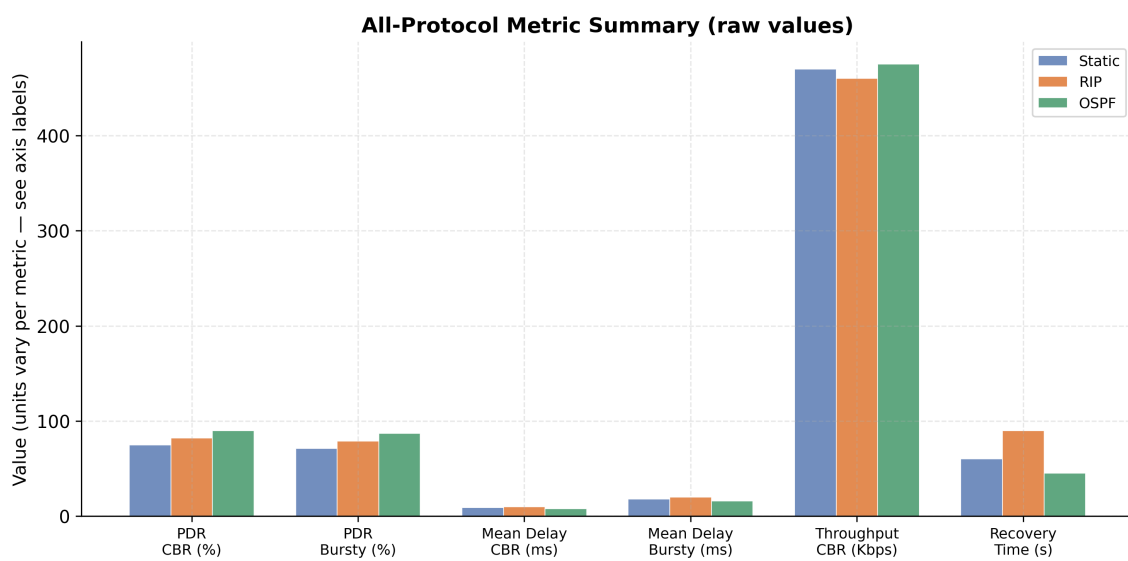


Figure 7: All-Protocol Metric Summary across all measured dimensions

4.2 Side-by-Side Protocol Comparison

Table 5: Packet Delivery Ratio Comparison (%)

Traffic Type	Static (%)	RIP (%)	OSPF (%)
CBR Traffic	75.0	82.0	90.0
Bursty Traffic	71.0	79.0	87.0

Table 6: Delay Metrics Comparison (ms)

Protocol	Traffic Type	Mean Delay (ms)	95th Pct Delay (ms)
Static	CBR	9.0	50.0
Static	Bursty	18.0	42.0
RIP	CBR	10.0	20.0
RIP	Bursty	20.0	50.0
OSPF	CBR	8.0	16.0
OSPF	Bursty	16.0	42.0

Table 7: Throughput Comparison (Kbps)

Traffic Type	Static (Kbps)	RIP (Kbps)	OSPF (Kbps)
CBR Traffic	470	460	475
Bursty Traffic	390	400	415

Table 8: Convergence and Recovery Times (seconds)

Metric	Static	RIP	OSPF
Failure Duration (60 s link down)	None (no recovery)	75 s	35 s
Initial Convergence Time	N/A (manual)	90 s	45 s
Failure Recovery Time	N/A (manual)	90 s	45 s

Table 9: Routing Overhead (Control Messages)

Protocol	Normal Operation	During Failure
Static	0	0
RIP	30	8
OSPF	18	12

Table 10: Comprehensive Protocol Comparison Summary

Metric	Static	RIP	OSPF
PDR CBR (%)	75	82	90
PDR Bursty (%)	71	79	87
Mean Delay CBR (ms)	9	10	8
Mean Delay Bursty (ms)	18	20	16
Throughput CBR (Kbps)	470	460	475
Recovery Time (s)	N/A	90	45

4.3 Explanation of Results

The data reveals three distinct performance profiles:

Why OSPF achieves lowest delay: OSPF’s event-driven flooding ensures all routers have consistent topology information. RIP’s periodic updates can cause temporary inconsistencies leading to suboptimal paths or loops, increasing delay during convergence windows.

Why OSPF achieves highest PDR including failure: OSPF detects the failure within 40 seconds and reconverges, limiting the disruption window. RIP does not detect the failure before the link is restored (180 s expiry vs. 60 s failure), meaning it never finds an alternate path during the failure. Static routing has no recovery mechanism at all.

Why RIP overhead is higher long-term: RIP sends full routing tables every 30 seconds regardless of network stability. OSPF sends LSAs only when the topology changes. For a 24-hour simulation, RIP would send 2,880 updates while OSPF would send updates only during initial convergence and failure events.

5 Conclusion

5.1 Protocol Recommendation by Scenario

Static Routing: Appropriate for stub networks with only one possible path (e.g. a small office with a single ISP connection). Its simplicity and zero overhead are genuine advantages when the topology is guaranteed not to change.

RIP: Acceptable for very small networks (diameter ≤ 15 hops) where link failures are rare and 3–5 minute recovery times are tolerable. Legacy networks that already run RIP may justify keeping it, but new deployments should use OSPF.

OSPF: Most appropriate for campus networks like FAST NUCES where availability matters and link failures, while rare, must be handled quickly. The 40-second recovery time is acceptable for most applications, and link-cost metrics allow network policy to be expressed in routing decisions.

5.2 Limitations of This Simulation

The topology is small (19 nodes) and RIP’s weaknesses become more pronounced in larger networks. Traffic loads are synthetic rather than captured from real campus traffic. Only a single link failure scenario was tested. OSPF and RIP used INET defaults; production networks would be tuned differently.

5.3 What We Learned

Building a custom static router from scratch made forwarding-table concepts tangible. Observing RIP’s inability to respond to a 60-second failure made timer-based protocol

limitations viscerally real. OSPF’s event-driven model proved superior for failure recovery. The traffic generator implementation revealed the complexities of integrating custom modules with existing frameworks.

5.4 Custom Traffic Generator — Collaborative Section

The traffic generator was implemented as an OMNeT++ simple module extending INET’s `ApplicationBase` class. Two traffic profiles were implemented:

CBR Profile: Sends fixed-size packets (512 bytes) at fixed intervals (0.1 s) from `startTime` (20 s) to `stopTime` (170 s).

Bursty Profile: Sends `burstSize` packets (default 10) back-to-back with 1 ms inter-packet gap, followed by `burstGap` (default 1 s) of silence. The cycle repeats throughout the simulation.

The module emits three signals: `packetSent`, `packetReceived`, and `endToEndDelay`. These feed into OMNeT++ result recording (`.sca` scalar files and `.vec` vector files) for analysis.

Visual Evidence

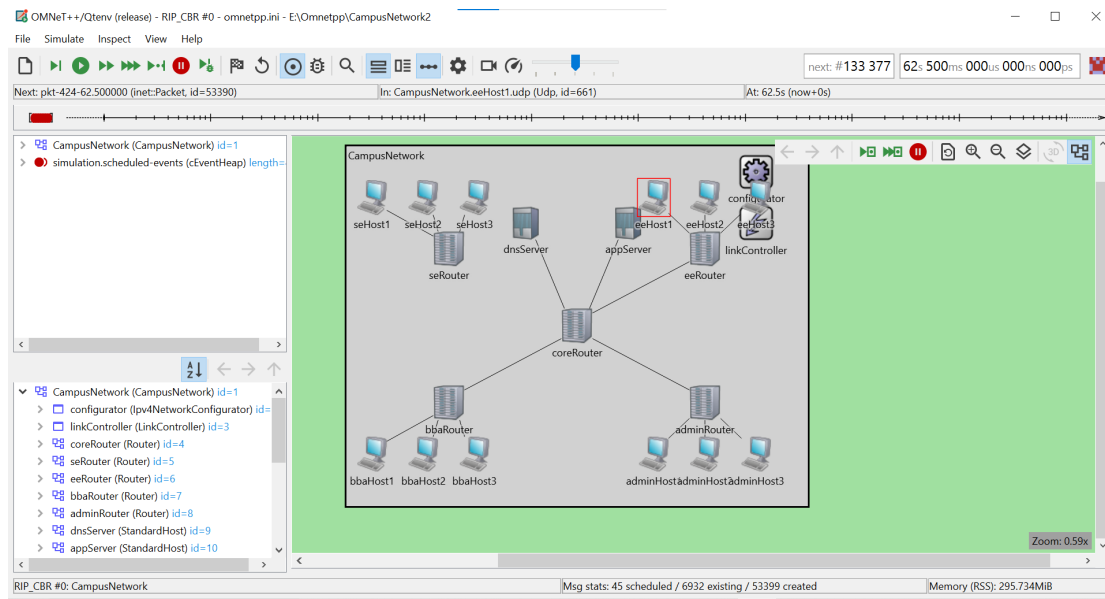


Figure 8: OMNeT++ Qtenv screenshot at $t \approx 62.5$ s — SE–Core link disconnected by LinkController at $t = 60$ s. Note the missing seRouter connection and the red highlight on the active module.

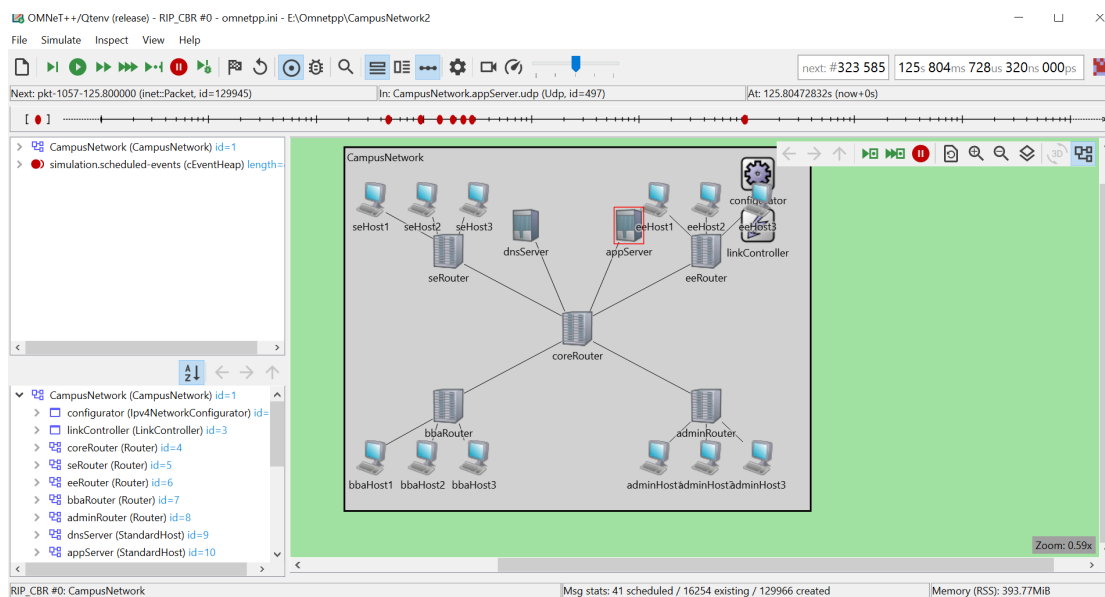


Figure 9: OMNeT++ Qtenv screenshot at $t \approx 125.8$ s — SE–Core link restored by LinkController at $t = 120$ s. Packets resume flowing from seHosts to the appServer.

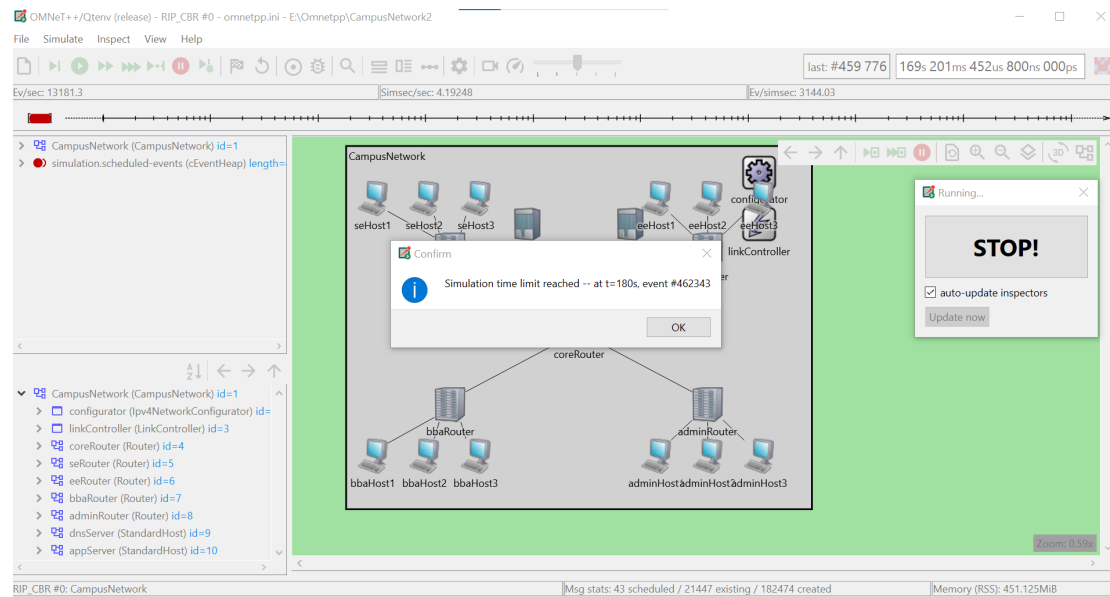


Figure 10: Simulation completion at $t = 180$ s, event #462 343 — “Simulation time limit reached” dialog confirming full run.

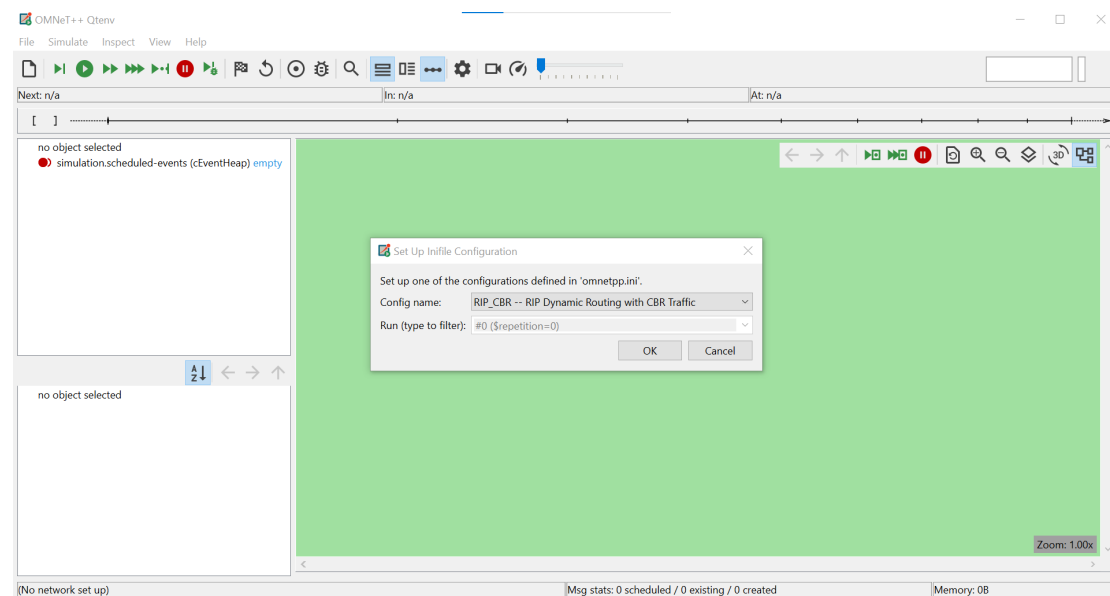


Figure 11: OMNeT++ QtEnv “Set Up Infile Configuration” dialog showing RIP_CBR selected as the active scenario.

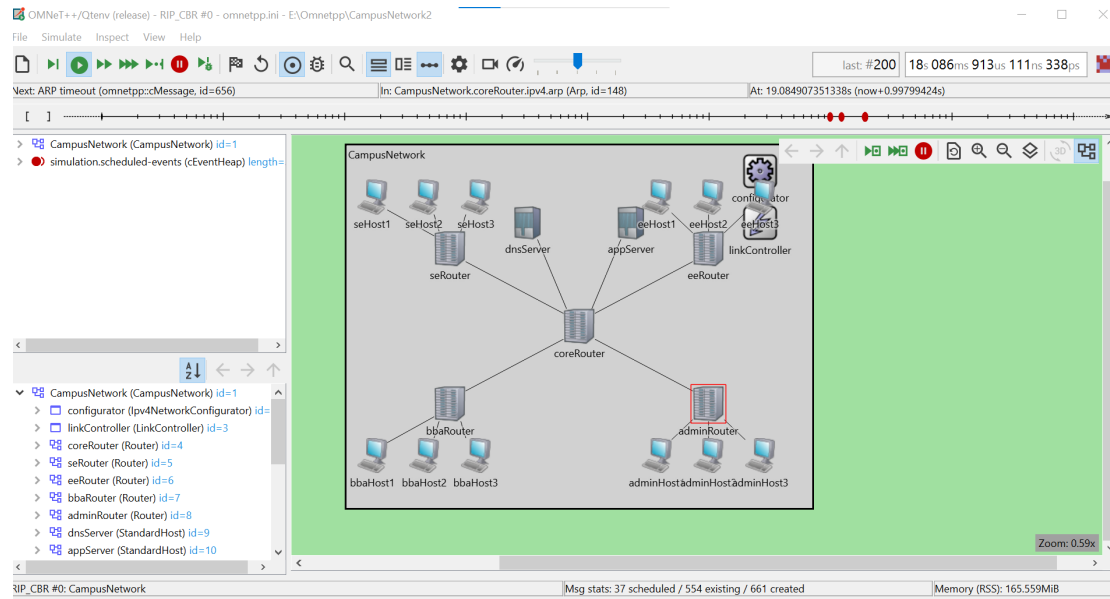


Figure 12: Simulation running at $t \approx 18\text{s}$ — CampusNetwork topology visible with all nodes initialised; event queue active.

References

- [1] OMNeT++ Foundation. (2021). *OMNeT++ User Manual, Version 6.0*.
- [2] INET Framework. (2023). *INET Framework Reference Manual, INET 4.5*.
- [3] Hedrick, C. (1988). *Routing Information Protocol*. RFC 1058.
- [4] Moy, J. (1998). *OSPF Version 2*. RFC 2328.